
Oliver Krauss

Data Structures I: Stack, Queue & Linked List

ALD | MTD.ba VZ SS26

Hagenberg 26.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Where We Left Off

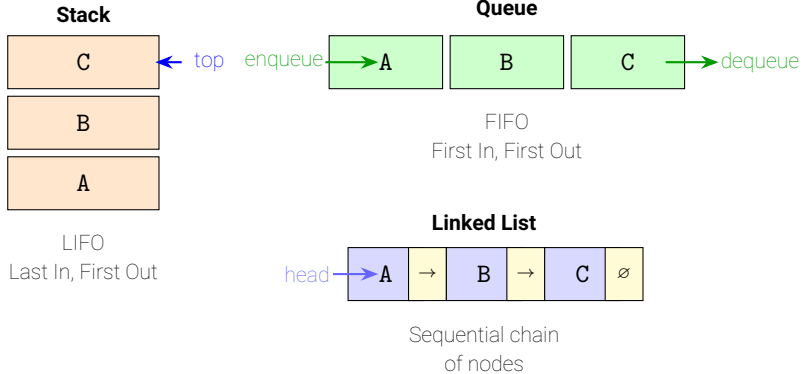
Previously we learned about interfaces, lambdas, and polymorphic collections -- today we build the structures that power them.

You implemented **Selectable** and **Transformable** interfaces and sorted shapes:

- **Interfaces**: **Selectable**, **Transformable** -- contracts without implementation
- **Lambda sort**: `shapes.sort(Comparator.comparing(Shape::getArea))`
- **Polymorphic array**: `Shape[]` holding mixed subtypes in your JavaFX canvas
- **Events**: lambda handlers for mouse/keyboard interactions

Today: we open the hood -- implement Stack, Queue, and LinkedList from scratch to understand *how* these structures work internally.

Three Structures: Overview



Where You Already Know These

Stack (LIFO)

- Browser **Back** button
- Editor **Undo** (Ctrl+Z)
- JVM **call stack**
- Expression evaluation

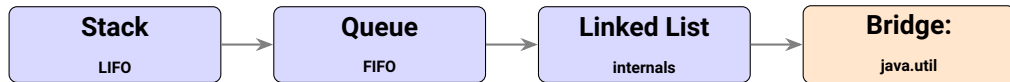
Queue (FIFO)

- **Print** queue
- Game **event** queue
- NPC **task** queue
- Thread pool work queue

Linked List

- Music **playlist**
- Text editor **cursor**
- OS process list

Today's Roadmap



What is a Stack?

A **Stack** enforces Last-In, First-Out (**LIFO**) access.

Three fundamental operations:

- `push(Object item)` -- add to the top
- `pop(): Object` -- remove from the top
- `peek(): Object` -- read top without removing

Invariant: you may only interact with the *top* element. No random access (you cannot ask for the 3rd item -- only the top). No middle-of-the-list operations.

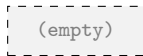
Classic analogy

A stack of plates: you add a new plate on top, and you take the top plate off first.

Browser **Back** button: each page you visit is pushed; pressing Back pops the most recent.

Stack Visualized: Push

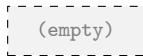
empty



top = -1

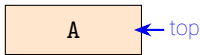
Stack Visualized: Push

empty



top = -1

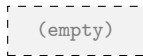
push(A)



top = 0

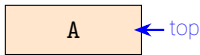
Stack Visualized: Push

empty



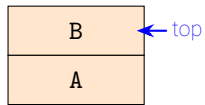
top = -1

push(A)



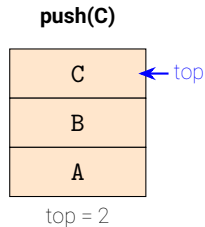
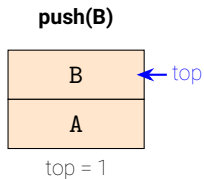
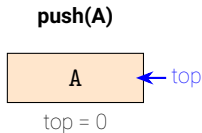
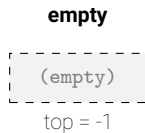
top = 0

push(B)

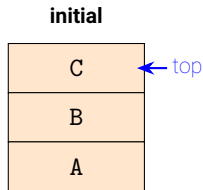


top = 1

Stack Visualized: Push



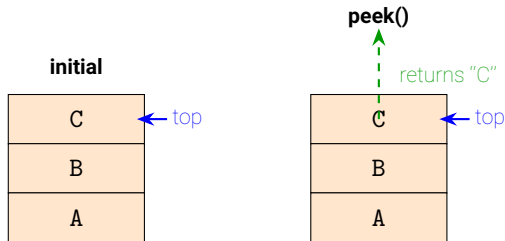
Stack Visualized: Pop & Peek



`peek()` looks, `pop()` removes

`peek()` returns the top value without changing the stack. **`pop()`** both returns and removes it.

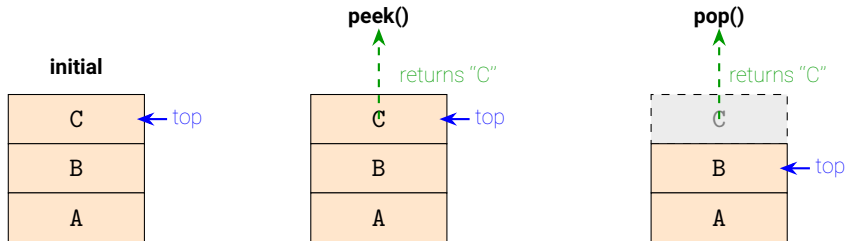
Stack Visualized: Pop & Peek



peek() looks, pop() removes

peek() returns the top value without changing the stack. pop() both returns and removes it.

Stack Visualized: Pop & Peek



peek() looks, pop() removes

peek() returns the top value without changing the stack. pop() both returns and removes it.

Example: Undo System

JavaFX drawing app: user action is a **DrawingCommand** managed via stack.

```
1  // DrawingCommand: execute() + undo()  -- full interface/class in Appendix
2
3  // In the controller:
4  Stack history = new Stack();  // our own Stack from this session
5
6  void onUserAction(DrawingCommand cmd) {
7      cmd.execute();             // perform the action
8      history.push(cmd);         // record it on the history stack
9  }
10
11 void onUndo() {
12     if (!history.isEmpty()) {
13         DrawingCommand cmd = (DrawingCommand) history.pop();
14         cmd.undo();             // reverse last action
15     }
16 }
```

Designing Stack: API First

Before implementing, define the contract:

Method	Return	Description
<code>push(Object item)</code>	<code>void</code>	Add item to top of stack
<code>pop()</code>	<code>Object</code>	Remove and return top item
<code>peek()</code>	<code>Object</code>	Return top item without removing
<code>isEmpty()</code>	<code>boolean</code>	True if stack contains no elements
<code>size()</code>	<code>int</code>	Number of elements currently on stack

We store **Object** references -- Java's root type

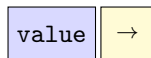
We use **Object** so one implementation works for any element type. **pop()** and **peek()** throw **NoSuchElementException** on an empty stack.

Later we learn about generics, so we can use specific classes for stack.

The Node: Building Block

Every linked structure is built from **nodes**:

```
1  class Node {  
2      Object value;  
3      Node  next;  
4  
5      Node(Object value) {  
6          this.value = value;  
7          this.next  = null;  
8      }  
9  }
```



one Node

Two-compartment visualization:
left = **value**, right = **next** pointer (→ or ∅).

A node stores one **value** and a reference to the **next** node.

Chain them together and you get a linked structure -- no array needed.

Implementing a Stack: Two Approaches

We need to store elements and remember which is on top. Two natural designs:

Approach A: Array-backed

Keep elements in an **Object[]**. Track the top with an **int** index.

push: store at **++top**

pop: read **data[top-]**

peek: read **data[top]**

Approach B: Node chain

Each item wraps the element and a pointer to the item below it.

push: new node, point to old top

pop: read top node, follow pointer

peek: read top node

Both are correct. Which is simpler? Which is faster?

Approach A: Array-backed Stack

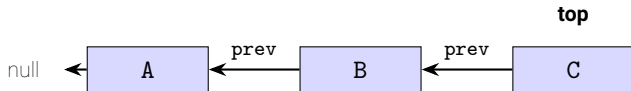
```
1  public class Stack {
2      private Object[] data = new Object[16];
3      private int top = -1;    // -1 means empty
4      public void push(Object item) {
5          // (resize if full, then:)
6          data[++top] = item;
7      }
8      public Object pop() {
9          Object result = data[top];
10         data[top--] = null; // null slot: let GC reclaim the object
11         return result;
12     }
13     public Object peek()    { return data[top]; }
14     public boolean isEmpty(){ return top == -1; }
15 }
```

Key detail: When the array is full we *grow*: allocate a new array of double the size and copy. This is a one-off $O(n)$ cost but it happens rarely.

Approach B: Node-chain Stack (Part 1)

Each element is wrapped in a **StackNode** that points to the node below it:

```
1  class StackNode {  
2      Object value;  
3      StackNode previous; // the node beneath this one  
4  
5      StackNode(Object value, StackNode previous) {  
6          this.value    = value;  
7          this.previous = previous;  
8      }  
9  }
```



Approach B: Node-chain Stack (Part 2)

```
1  public class Stack {
2      private StackNode top = null;  // null means empty
3
4      public void push(Object item) {
5          top = new StackNode(item, top);  // new node sits on top
6      }
7
8      public Object pop() {
9          Object result = top.value;
10         top = top.previous;           // drop the top node
11         return result;
12     }
13
14     public Object peek()    { return top.value; }
15     public boolean isEmpty(){ return top == null; }
16 }
```

Your Turn: Approach A vs B Trade-offs

Discuss with a neighbour -- fill in the table:

	Array-backed	Node chain
push		
pop / peek		
Memory per element		
Memory allocation		
Cache performance		

Hint: think about what happens when the array is full. And where in memory do node objects end up?

Approach A vs B: Trade-offs -- Answer

	Array-backed	Node chain
push	$O(1)$ amortized*	$O(1)$ always
pop / peek	$O(1)$	$O(1)$
Memory per element	1 slot (compact)	1 node + pointer (overhead)
Memory allocation	bulk (resize rarely)	1 allocation per push
Cache performance	good (contiguous)	worse (scattered in heap)

*Amortized $O(1)$: a grow copies n elements but doubles capacity, so it costs at most 1 extra "token" per prior push on average -- still $O(1)$ overall.

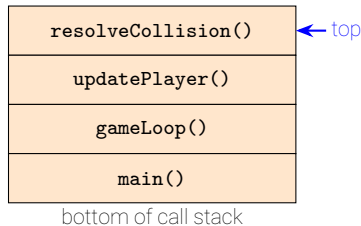
In practice

Array-backed (like Java's **ArrayDeque**) is preferred: better cache locality, less allocation overhead.

The Call Stack

The JVM maintains a **call stack** -- the same data structure you just built.

- Each method call = **push()** a *stack frame*
- Each method return = **pop()** the frame
- Stack traces in exceptions show the stack at crash time
- **StackOverflowError** = stack is full (infinite recursion)



Your debugger IS showing you a Stack

The stack trace panel in your IDE is exactly the **toString()** of the JVM call stack. Every time you step through code in the debugger, you are navigating this structure.

Recap: Stack

- **LIFO**: Last In, First Out -- only the top is accessible
- **Three operations**: `push`, `pop`, `peek`
- **Array-backed**: `Object[]` with a `top` index pointer
- **$O(1)$ amortized**: `push`/`pop`/`peek` all constant time
- **Type erasure**: `new T[n]` is illegal; use `Object[]` + cast
- **Null the slot**: after `pop()`, set `data[top] = null` to prevent memory leaks
- **Use cases**: undo, browser Back, call stack, expression evaluation, DFS
- **In stdlib**: `ArrayDeque<T>` (not `java.util.Stack`)

Any questions before we move to Queue?

What is a Queue?

A **Queue** enforces First-In, First-Out (**FIFO**) access.

Three fundamental operations:

- `enqueue(Object item)` -- add to the *back*
- `dequeue(): Object` -- remove from the *front*
- `peek(): Object` -- read front without removing

Key distinction from Stack:

You add at one end and remove from the **other**.

Classic analogy

A checkout queue at a supermarket.
The first person in line is served first.
New customers join at the back.

Add at one end, remove from the
OTHER

This single rule distinguishes Queue
from Stack.

Queue Visualized

Step

Queue state

start

(empty)

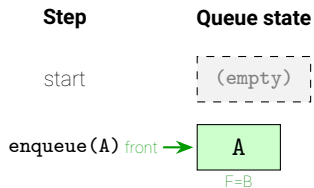
Rules:

- **enqueue**: add to **back**
- **dequeue**: remove from **front**
- F = front pointer, B = back pointer

FIFO

First element in is the first out -- front and back move independently.

Queue Visualized



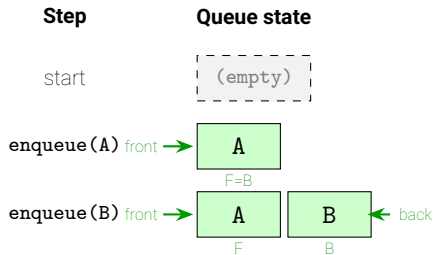
Rules:

- enqueue: add to **back**
- dequeue: remove from **front**
- F = front pointer, B = back pointer

FIFO

First element in is the first out -- front and back move independently.

Queue Visualized



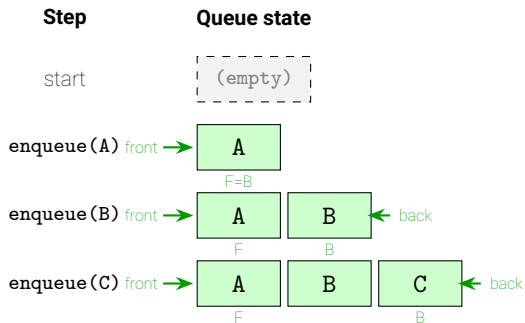
Rules:

- enqueue: add to **back**
- dequeue: remove from **front**
- F = front pointer, B = back pointer

FIFO

First element in is the first out -- front and back move independently.

Queue Visualized



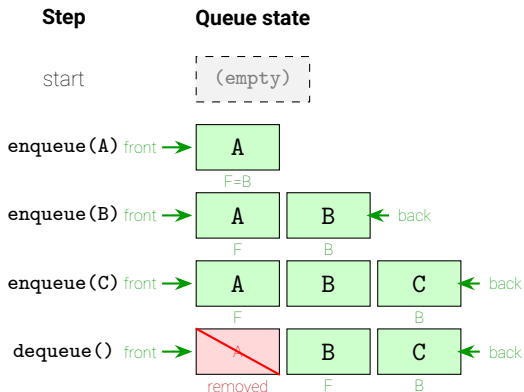
Rules:

- enqueue: add to **back**
- dequeue: remove from **front**
- F = front pointer, B = back pointer

FIFO

First element in is the first out -- front and back move independently.

Queue Visualized



Rules:

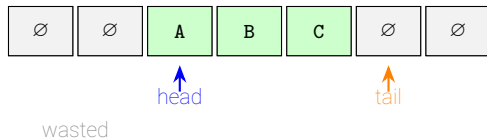
- enqueue: add to **back**
- dequeue: remove from **front**
- F = front pointer, B = back pointer

FIFO

First element in is the first out -- front and back move independently.

head + tail Pointers

A naive array-backed Queue shifts all elements on dequeue ($O(n)$). Two pointers solve this:



- **head**: index of the front element
- **tail**: index where next element will be added
- **Dequeue**: just increment **head** (no shifting!)
- **Enqueue**: store at **tail**, increment **tail**

Wasted space

As head advances, slots $0..head-1$ become unusable. Solution: **circular buffer** (Appendix).

The diagram shows the state after two dequeues: head started at 0 (pointing to A), advanced to 1 (pointing to B), then to 2 (now pointing to C). Slots 0 and 1 are the wasted space.

Array-Backed Queue: enqueue() + dequeue()

Fields: `Object[] data; int head=0, tail=0, size=0;`

```
1 // enqueue: store at tail, advance
2 public void enqueue(Object item) {
3     data[tail] = item;
4     tail++;
5     size++;
6 }
```

```
1 // dequeue: read head, advance
2 public Object dequeue() {
3     if (size == 0)
4         throw new
5             NoSuchElementException();
6     Object v = data[head];
7     data[head] = null; // allow GC
8     head++;
9     size--;
10    return v;
11 }
```

head and **tail** only advance -- slots before **head** become wasted

This is the simple version. Production queues wrap around using modular arithmetic (circular buffer -- see Appendix). That is exactly how **ArrayDeque** works internally.

Queue: Two Backing Approaches

Array-backed Queue

- Pre-allocate a fixed array
- **head** index advances on dequeue
- **tail** index advances on enqueue
- Slots before **head** become wasted (solved with a **circular buffer**)
- **Cache-friendly** -- contiguous memory

Node-chain Queue

- Each element is a **Node** on the heap
- **head** → front (dequeue here)
- **tail** → back (enqueue here)
- No wasted slots, grows without limit
- **Extra memory** per node (object header + **next** pointer)

Both give $O(1)$ enqueue and dequeue

The practical exercise (UE6) asks you to implement the node-chain version from scratch.

Queue: Performance & Space

	Array-backed	Node-chain
enqueue	$O(1)$ amortized	$O(1)$
dequeue	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
Memory per element	1 array slot	node + pointer overhead
Wasted space	possible (before head)	none
Cache locality	✓ contiguous	✗ scattered on heap
Resizing	copy to larger array	never needed

Rule of thumb

Bounded or high-throughput? → **Array** (**ArrayDeque**).

Unbounded or frequent insert/delete? → **Node-chain** (**LinkedList**).

Stack vs. Queue: Side-by-Side

Stack (LIFO)

- Add/remove from **same** end (top)
- **push** / **pop** / **peek**
- Array-backed with **top** index
- **$O(1)$** amortized all ops
- Use: undo, Back, call tracking, DFS
- stdlib: **ArrayDeque** (push/pop API)

Queue (FIFO)

- Add at back, remove from **front**
- **offer** / **poll** / **peek**
- Node-backed (LinkedList) or circular buffer
- **$O(1)$** all ops
- Use: event queue, print queue, BFS
- stdlib: **ArrayDeque** (offer/poll API)

Recap: Queue

- **FIFO**: First In, First Out -- process in the order received
- **Three operations**: **enqueue** (offer), **dequeue** (poll), **peek**
- **head + tail**: two pointers avoid shifting; head advances on dequeue
- **Linked-backed**: $O(1)$ both ends, no wasted space, no circular logic needed
- **In stdlib**: `Queue<T>` interface, `ArrayDeque<T>` implementation
- **Use cases**: event queues, print queues, BFS graph traversal (S7)

Any questions before we open the hood on Linked List?

The “Why” of Linked Lists

Contiguous memory (Array)

Elements stored side-by-side. Insert at front = shift all elements right. Very cache-friendly for reads, but costly for front-insertions.

Scattered nodes (Linked List)

Each element stores a *pointer* to the next. Insert at front = change two pointers. No shifting at all.

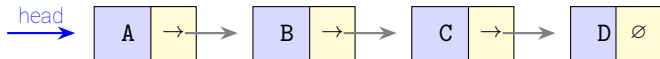
The fundamental trade-off

Array: $O(1)$ random access, $O(n)$ front insert.

Linked List: $O(1)$ front insert, $O(n)$ random access (must traverse from head).

There is no free lunch -- choose based on your usage pattern.

Singly Linked: Structure



$O(1)$ front insert

New node: set **next** = **head**,
update **head**. Two
assignments, no shifting.

$O(n)$ random access

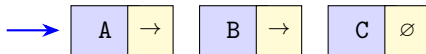
To get element **i**, traverse
from **head** through **i** nodes.
No index shortcut.

Traversal

```
current = head  
while (current != null)  
    current = current.next
```

Singly Linked: addFirst()

initial: head → **A** → **B** → **C**

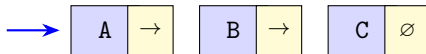


```
1 void addFirst(Object value) {  
2     Node node = new Node(value);  
3     node.next = head;  
4     head = node; }
```

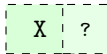
$O(1)$: two pointer assignments, no shifting

Singly Linked: addFirst()

initial: head $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C



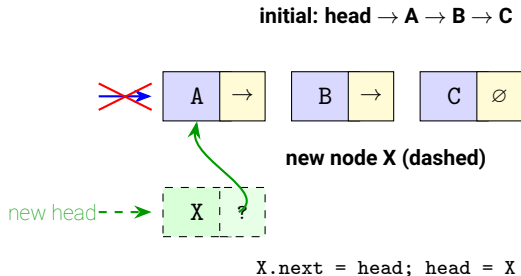
new node X (dashed)



```
1 void addFirst(Object value) {  
2     Node node = new Node(value);  
3     node.next = head;  
4     head = node; }
```

$O(1)$: two pointer assignments, no shifting

Singly Linked: addFirst()



```
1 void addFirst(Object value) {  
2     Node node = new Node(value);  
3     node.next = head;  
4     head = node; }
```

$O(1)$: two pointer assignments, no shifting

Singly Linked: traversal pattern

```
1  // Traversal pattern
2  Node current = head;
3  while (current != null) {
4      // process current.value
5      current = current.next;
6  }
```

ALWAYS the same pattern

Every linked list algorithm -- search, print, sum, copy -- uses this while loop.

Singly Linked: why **addLast** is slower

Key idea

In a singly-linked list you can only move forward, so to append you must **walk to the end** first.

Complexity

- **addFirst**: $O(1)$ (just change **head**)
- **addLast**: $O(n)$ (must traverse)

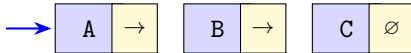
Fix

Keep an extra pointer **tail** to the last node.

Then **addLast** becomes $O(1)$.

Singly Linked: remove()

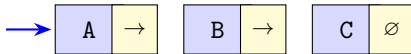
before: A → B → C (remove B)



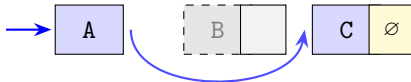
```
1  Node prev = null; Node curr = head;
2  while (curr != null && !curr.value.equals(x)) { prev = curr; curr = curr.next; }
3  if (curr != null) {
4      if (prev == null) head = curr.next;    // removing head
5      else prev.next = curr.next;           // bypass the node
6  }
```

Singly Linked: remove()

before: A → B → C (remove B)

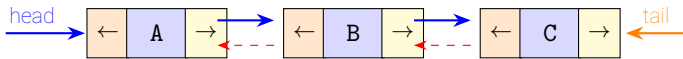


after: A.next = B.next



```
1 Node prev = null; Node curr = head;
2 while (curr != null && !curr.value.equals(x)) { prev = curr; curr = curr.next; }
3 if (curr != null) {
4     if (prev == null) head = curr.next; // removing head
5     else prev.next = curr.next;         // bypass the node
```

Doubly Linked: Structure



Singly-Linked

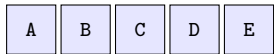
<code>addFirst</code>	$O(1)$
<code>addLast</code>	$O(n)$
<code>removeFirst</code>	$O(1)$
<code>removeLast</code>	$O(n)$

Doubly-Linked (with **tail**)

<code>addFirst</code>	$O(1)$
<code>addLast</code>	$O(1) \checkmark$
<code>removeFirst</code>	$O(1)$
<code>removeLast</code>	$O(1) \checkmark$

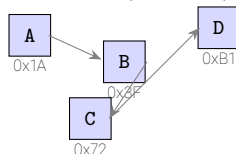
Memory Layout: Array vs. Linked

Array (contiguous)



cache-friendly: all in one block

Linked (scattered)



Operation	Array / ArrayList	LinkedList (doubly)
Random access <code>get(i)</code>	$O(1)$ ✓	$O(n)$
Insert at front	$O(n)$	$O(1)$ ✓
Insert at back	$O(1)$ amortized	$O(1)$ ✓
Remove from front	$O(n)$	$O(1)$ ✓

Array's cache locality often beats LinkedList's $O(1)$ in practice

Modern CPUs prefetch contiguous memory. Traversing a linked list has poor cache performance. Always benchmark before choosing.

Recap: Linked List

- **Node**: value + next (+ prev for doubly); **head** is the entry point
- **Singly-linked**: $O(1)$ `addFirst`, $O(n)$ `addLast/removeLast`
- **Doubly-linked**: adds `prev` pointer and `tail` reference -- $O(1)$ both ends
- **head + tail**: two pointer assignments for `addFirst/addLast` -- drawing helps
- **Traversal**: `current = head; while (current != null) { current = current.next; }`
- `java.util.LinkedList`: doubly-linked; preferred as Deque, rarely as plain List
- **Cache locality**: arrays beat linked lists in traversal speed despite same Big-O

Any questions before we put it all together?

Quick Retrieval: Test Yourself

Before looking at the summary -- can you answer these from memory?

1. What does **LIFO** mean? Give one real-world example.
2. Why can an **array-backed stack** do **push/pop/peek** in (amortized) $O(1)$?
3. Why does a **queue** conceptually need a **front** and a **back**? What goes wrong if you only track one end?
4. Compare **array-backed** vs. **node-chain** queues: when is each the better choice?
5. Why is removing from the **front** cheap in a linked structure but expensive in a contiguous array?
Hint: what must be moved/updated?

Take some time to think, then we go through the answers.

Quick Retrieval: Answers

1. **Last In, First Out.** Example: browser Back button -- the most recently visited page is returned to first.
2. **The top is a single index.** **push** writes to the next slot and increments; **pop** decrements and reads.
Amortized $O(1)$ because resizing is rare (copy happens only when capacity is exceeded).
3. **FIFO needs two ends:** enqueue at the **back**, dequeue at the **front**.
With only one pointer, you'd either lose where to insert next, or you'd have to **shift** elements after every dequeue ($O(n)$).
4. **Array-backed (ArrayDeque):** contiguous, cache-friendly, very fast; may need resizing / wrap-around bookkeeping.
Node-chain (LinkedList): no pre-allocation, easy growth; higher memory overhead and worse cache locality.
5. **Linked:** remove-first is a constant number of pointer updates ($O(1)$).
Array: removing index 0 requires shifting the remaining $n - 1$ elements to keep them contiguous ($O(n)$).

Summary

Stack

- LIFO: only touch the top
- **push** / **pop** / **peek**
- Array-backed, **top** index
- **$O(1)$** amortized all ops
- Null the slot after pop!
- stdlib: **ArrayDeque**

Queue

- FIFO: add back, remove front
- **offer** / **poll** / **peek**
- head + tail pointers
- **$O(1)$** all ops
- BFS in S7
- stdlib: **ArrayDeque**

Linked List

- Node: value + next
- Doubly: + prev + tail
- **$O(1)$** front/back inserts
- **$O(n)$** random access
- Traversal: while loop
- stdlib: **LinkedList**

Backup: Big-O Comparison Table

Operation	Array	ArrayList	Stack	Queue	Linked (single)	Linked (double)
Random access	$O(1)$	$O(1)$	--	--	$O(n)$	$O(n)$
Insert front	$O(n)$	$O(n)$	--	--	$O(1)$	$O(1)$
Insert back	--	$O(1)^*$	$O(1)^*$	$O(1)$	$O(n)$	$O(1)$
Remove front	$O(n)$	$O(n)$	--	$O(1)$	$O(1)$	$O(1)$
Remove back	--	$O(1)$	$O(1)$	--	$O(n)$	$O(1)$
Push / Enqueue	--	--	$O(1)^*$	$O(1)$	--	--
Pop / Dequeue	--	--	$O(1)$	$O(1)$	--	--

* amortized $O(1)$ -- occasional resize is $O(n)$ but averages out

Color guide

Green cells = best choice for that operation. Choose your structure by your *dominant operation*.

Backup: Circular Buffer Queue

The wasted-space problem with head+tail pointers is solved by wrapping around:



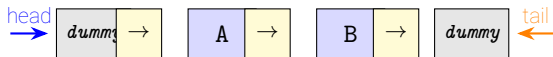
```
1 // Enqueue with wrapping:
2 data[tail] = item;
3 tail = (tail + 1) % capacity; // wrap around
4
5 // Dequeue with wrapping:
6 Object value = data[head];
7 data[head] = null;
8 head = (head + 1) % capacity; // wrap around
```

This is how **ArrayDeque** actually works internally

No wasted space, no node allocation overhead -- pure array efficiency with **O(1)** both ends.

Backup: Sentinel Nodes

A **sentinel** (dummy) head and tail eliminate special-case logic for empty lists:



Without sentinels: every operation checks `if (head == null)` -- easy to forget.

With sentinels: `head.next` is always valid; `tail.prev` is always valid. No null checks needed in `addFirst/addLast`.

`java.util.LinkedList` uses sentinel nodes internally

The JDK source code shows `first` and `last` are sentinel-like fields. This pattern is common in production-grade linked list implementations.

Backup: Stack with Linked Backing

```
1  public class LinkedStack {
2      private Node top;    // head of the node chain
3      private int  size;
4      public void push(Object item) {
5          top = new Node(item, top); // prepend -- O(1)
6          size++;
7      }
8      public Object pop() {
9          if (top == null) throw new NoSuchElementException();
10         Object value = top.value;
11         top = top.next; // advance -- GC handles old node
12         size--;
13         return value;
14     }
15     public Object peek()    { return top == null ? null : top.value; }
16     public int    size()    { return size; }
17     public boolean isEmpty() { return top == null; }
18 }
```

Backup: Deque Method Reference

Deque is a double-ended queue -- supports both Stack and Queue semantics:

Operation	Stack naming	Queue naming
Add to front	push(e)	offerFirst(e)
Add to back	(n/a)	offer(e) / offerLast(e)
Remove from front	pop()	poll() / pollFirst()
Remove from back	(n/a)	pollLast()
Peek front	peek()	peek() / peekFirst()
Peek back	(n/a)	peekLast()

One class, two personalities

ArrayDeque is the recommended implementation for both. Use **push/pop** when thinking about a stack; use **offer/poll** when thinking about a queue -- the method names communicate intent to readers.